

PROJECT REPORT

AI POWERED DEBT RELIEF & FINANCIAL RECOVERY PLATFORM

An Intelligent Solution for Smarter Debt Management,
Financial Freedom & Sustainable Recovery



TEAM DETAILS

Team ID	: SWTID-2026-2312
Team Leader	: Nitesh Chaudhary
Team member	: Naitik Sharma
Team member	: Sudhanshu Sharma
Team member	: Harsh Saini
Team member	: Manish Jangid



Project Description:

The **AI Powered Debt Relief & Financial Recovery Platform** is an intelligent AI-powered web application designed to simplify and automate debt management and financial recovery for borrowers. The platform helps users manage loan information, analyze financial health, generate AI-powered negotiation strategies, and receive personalized settlement recommendations through a secure and user-friendly digital platform.

The application is developed using **React.js** for the frontend, **FastAPI (Python)** for the backend, **SQLite** as the relational database, **SQLAlchemy ORM** for database operations, and **Google Gemini 2.5 Flash API** for AI-powered financial guidance and negotiation letter generation. **JWT** authentication ensures secure user access, while deployment is performed using **Vercel** (Frontend) and **Render** (Backend).

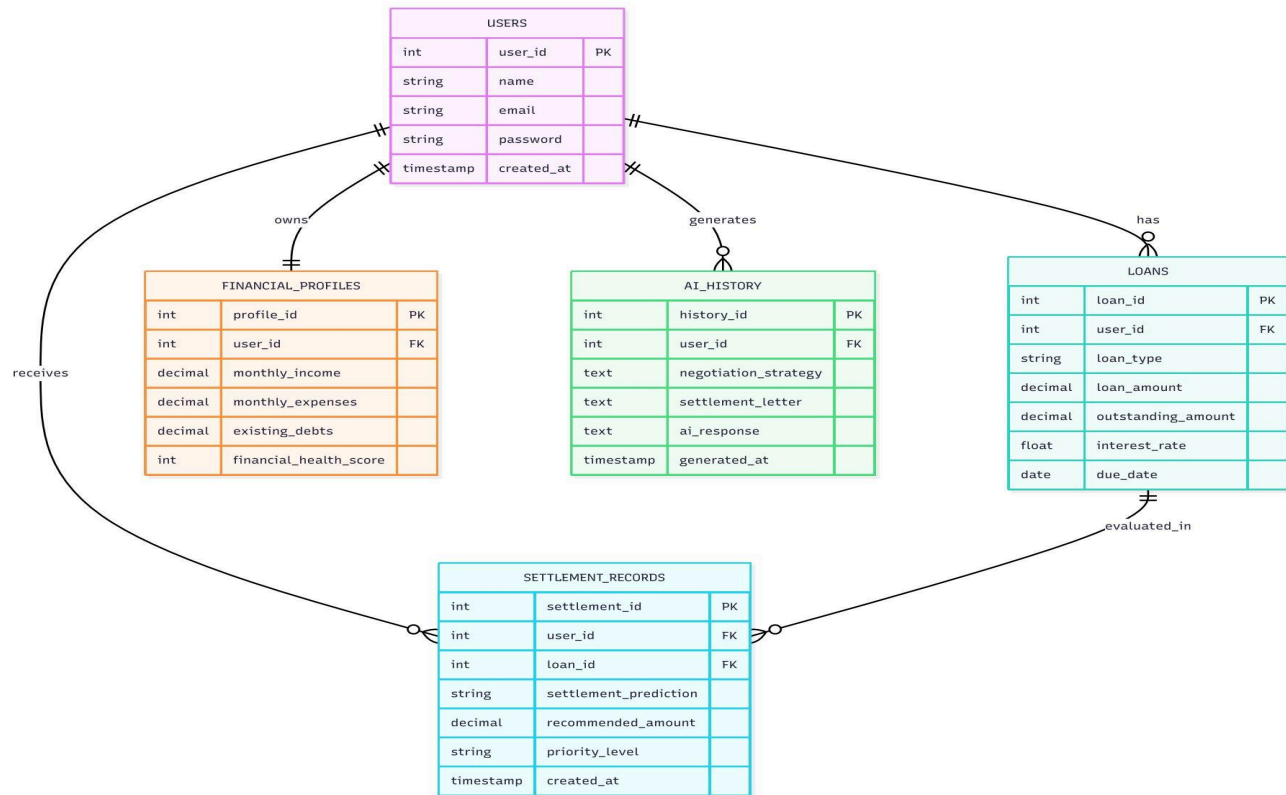
The platform enables borrowers to better understand their financial situation, reduce debt stress, and improve communication with lenders through AI-assisted recommendations.

Scenario 1: A borrower logs into the platform and enters loan details such as outstanding balance, monthly EMI, interest rate, overdue duration, and monthly income. The system analyzes the financial profile and calculates the EMI-to-Income ratio, debt stress level, and repayment capability. Based on this analysis, the AI engine generates personalized settlement recommendations and financial recovery guidance.

Scenario 2: The borrower selects a loan account and chooses a negotiation strategy. Using Google Gemini 2.5 Flash, the platform generates a professional lender-specific negotiation letter, including interest rate reduction requests, debt validation requests, or partial settlement proposals according to the borrower's financial condition.

Scenario 3: The borrower accesses the dashboard to monitor financial health indicators such as monthly surplus, EMI ratio, debt stress level, settlement percentage, and AI-generated recommendations. All negotiation letters and settlement history are stored for future reference.

Technical Architecture:



Description: The AI Powered Debt Relief & Financial Recovery Platform follows a modern client-server architecture that integrates Artificial Intelligence with financial data analysis. The frontend is developed using React.js and communicates with the FastAPI backend through REST APIs. SQLAlchemy ORM manages all database operations with SQLite, while Google Gemini 2.5 Flash generates AI-powered negotiation letters and settlement recommendations.

Pre-requisites:

- **React.js Framework Knowledge:** React Documentation
- **FastAPI Framework Knowledge:** FastAPI Documentation
- **Google Gemini API Familiarity:** Google AI Studio Documentation
- **SQLite Database Knowledge:** SQLite Documentation
- **SQLAlchemy ORM Knowledge:** SQLAlchemy Documentation
- **JWT Authentication Knowledge:** FastAPI Security Documentation
- **HTML, CSS, and JavaScript Skills:** W3Schools HTML/CSS/JavaScript Tutorials
- **Python Programming Proficiency:** Python Documentation
- **Version Control with Git:** Git Documentation
- **Development Environment Setup:** React.js, FastAPI, Python, and Node.js Installation Guide
- **Vercel for Frontend Deployment:** Vercel Documentation
- **Render for Backend Deployment:** Render Documentation

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Generate a Google Gemini API key and configure it securely in the project environment.
- **Activity 1.2:** Research and select the appropriate Generative AI model from Google Gemini for financial analysis, settlement recommendations, and negotiation letter generation.
- **Activity 1.3:** Define the architecture of the application, detailing interactions between the React frontend, FastAPI backend, SQLite database, SQLAlchemy ORM, JWT authentication, and Google Gemini API.
- **Activity 1.4:** Set up the development environment by installing the required libraries, frameworks, and dependencies for React.js, FastAPI, SQLite, SQLAlchemy ORM, and Google Gemini API.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the core functionalities including User Authentication, Loan Management, Financial Health Analysis, AI Settlement Recommendation, AI Negotiation Letter Generation, and Negotiation History.
- **Activity 2.2:** Implement the FastAPI backend to manage REST APIs, user authentication, financial calculations, SQLite database operations, and Google Gemini API integration.

Milestone 3: Backend Development

Activity 3.1: Write the main backend application by implementing API routes for user authentication, loan management, financial analysis, AI settlement recommendation, negotiation letter generation, database operations, and history management.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the user interface using React.js, Vite, Tailwind CSS, and Context API, ensuring a responsive and intuitive dashboard.
- **Activity 4.2:** Create dynamic user interfaces for loan management, financial health analysis, AI settlement recommendations, negotiation letter generation, and history tracking through REST API communication.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for deployment by configuring the frontend, backend, database, environment variables, and production settings.
- **Activity 5.2:** Deploy the React frontend on Vercel and the FastAPI backend on Render, ensuring secure communication between the frontend, backend, database, and Google Gemini API.

Milestone 6: Conclusion

The AI Powered Debt Relief & Financial Recovery Platform successfully combines Artificial Intelligence with modern web technologies to simplify debt management and financial recovery. The project provides borrowers with secure loan management, financial health analysis, AI-powered settlement recommendations, and professional negotiation letters. Its scalable architecture, intelligent automation, and user-friendly interface make it an efficient solution for improving financial decision-making and debt recovery assistance.

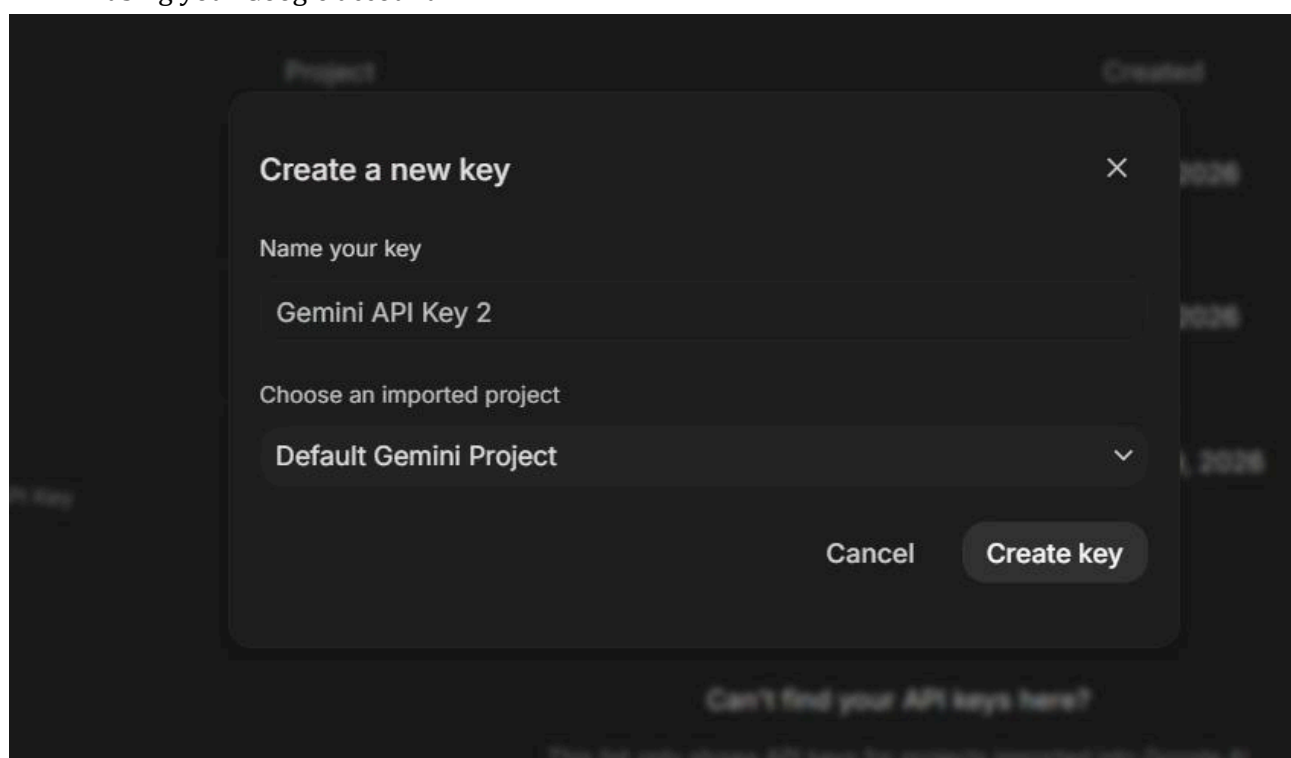
Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate Generative AI model for financial analysis, settlement recommendations, and negotiation letter generation. This involves researching the capabilities and performance of various AI models, selecting the most suitable model, defining the overall system architecture, and preparing the complete development environment required for building the AI Powered Debt Relief & Financial Recovery Platform.

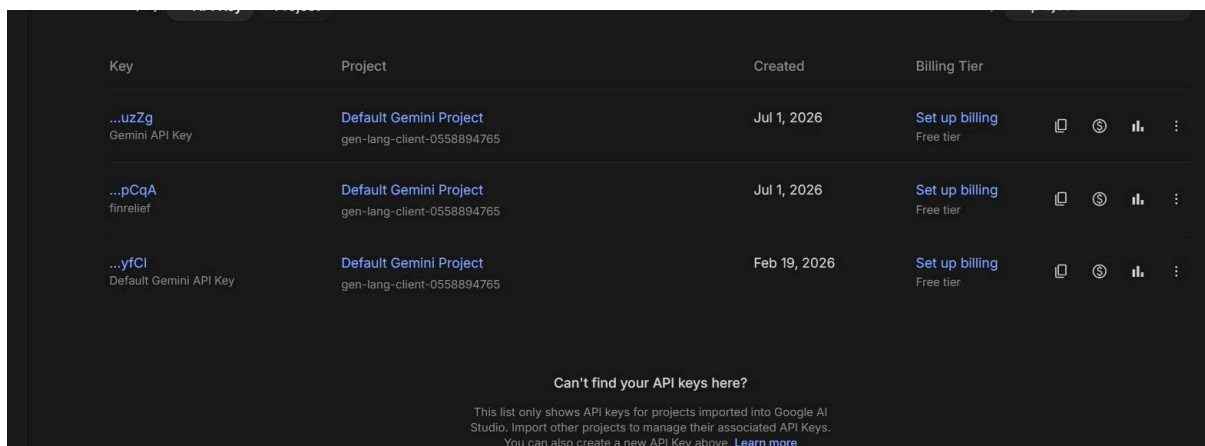
Activity 1.1: Generate a Google Gemini API Key

Before the application can communicate with the Google Gemini AI model, a valid Google Gemini API key is required. Follow the steps below to generate and configure the API key securely.

- **Step 1 — Create a Google AI Studio Account:** Visit <https://aistudio.google.com/> and sign in using your Google account.



- **Step 2 — Navigate to API Keys:** After signing in, open the API Keys section from the Google AI Studio dashboard.



- **Step 3 — Create a New API Key:**
Click Create API Key and generate a new API key for your project.
- **Step 4 — Copy the API Key:**
Copy the generated API key immediately and store it securely. Do not share it publicly.
- **Step 5 — Add the Key to Your Project:** Create a .env file in the backend project directory and add the API key.
- **Step 6 — Configure the Environment Variables:**
Load the API key securely in the FastAPI backend using environment variables.
- **Important:** Never upload your .env file or API key to GitHub. Add the .env file to .gitignore before pushing your project.

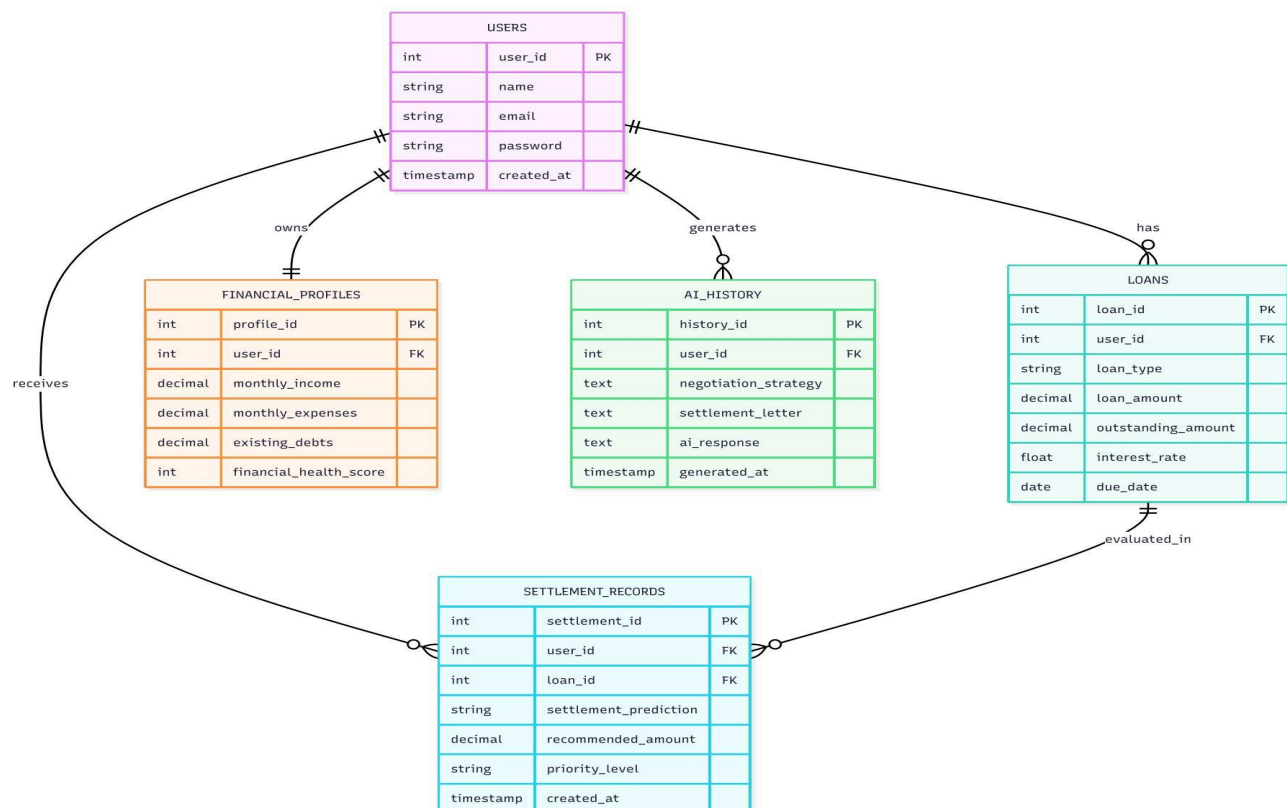
Activity 1.2: Research and Select the Appropriate Generative AI Model

Here are the important factors considered while selecting the AI model for the project.

- Understand the project requirements by analyzing financial recovery, debt settlement, negotiation letter generation, and borrower assistance.
- Explore Google Gemini documentation to compare available AI models, response quality, token limits, and API performance.
- Evaluate different AI models based on financial reasoning, document generation quality, response speed, and contextual understanding.
- Select **Google Gemini 2.5 Flash** because it provides fast inference, high-quality text generation, excellent contextual understanding, and efficient API integration for financial applications.

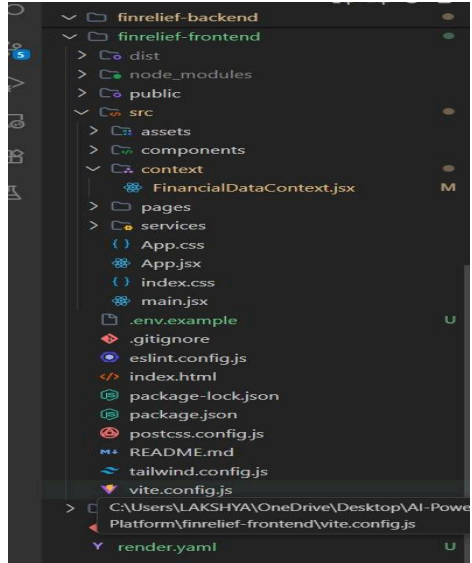
Activity 1.3: Define the Architecture of the Application

- Draft the system architecture showing the interaction between the React frontend, FastAPI backend, SQLite database, SQLAlchemy ORM, JWT authentication, and Google Gemini API.
- Define the frontend functionality where borrowers can register, manage loans, analyze financial health, request settlement recommendations, and generate AI negotiation letters.
- Define backend responsibilities including user authentication, loan management, financial calculations, AI request processing, database management, and REST API services.
- Describe the AI integration process where borrower financial information is analyzed and Google Gemini generates personalized settlement recommendations and professional negotiation letters.

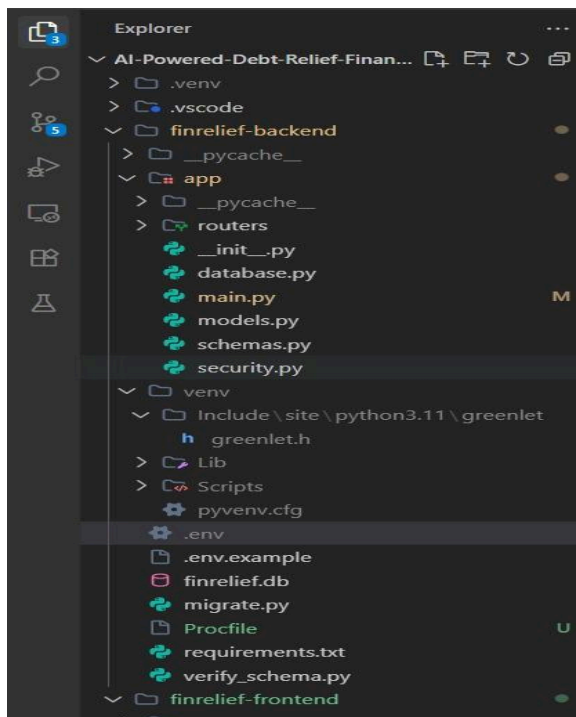


Activity 1.4: Set Up the Development Environment

- Install Python 3.11 and Node.js.
- Create separate frontend and backend project environments.



- Install React.js project dependencies.



- Configure SQLite database and SQLAlchemy ORM.
- Configure JWT Authentication.
- Configure the Google Gemini API key inside the `.env` file.
- Create the project structure containing frontend, backend, database models, API routes, authentication modules, AI services, and deployment configuration.

Milestone 2: Core Functionalities Development

Activity 2.1: Develop the Core Functionalities

The following core functionalities were implemented as part of the platform:

- **User Authentication:** Secure user registration and login using JWT Authentication.
- **Loan Management:** Users can add, update, delete, and manage loan information including creditor details, outstanding balance, EMI, interest rate, and due dates.
- **Financial Health Analysis:** Calculates EMI-to-Income Ratio, Monthly Surplus, Debt Stress Level, and Financial Health Score based on borrower financial information.
- **AI Settlement Recommendation:** Generates personalized debt settlement recommendations using Google Gemini 2.5 Flash.
- **AI Negotiation Letter Generation:** Generates professional negotiation letters such as Interest Rate Reduction Request, Debt Validation Request, and Partial Payment Settlement letters.
- **Negotiation History:** Stores all AI-generated negotiation letters and recommendations for future reference.

```
1 import os
2 from fastapi import FastAPI
3 from fastapi.middleware.cors import CORSMiddleware
4 from dotenv import load_dotenv
5
6 # Load environmental variables first
7 load_dotenv()
8
9 from app.database import engine
10 from app import models
11 from app.routers import auth, debts, profile, ai
12
13 # Initialize DB tables (SQLite auto-bootstrap)
14 models.Base.metadata.create_all(bind=engine)
15
16 app = FastAPI(
17     title="FinRelief AI Backend",
18     description="Secure backend service for debt recovery and analysis modules, authenticated via JWT and integrated with Google G",
19     version="1.0.0"
20 )
21
22 # Build allowed origins: always include localhost dev origins,
23 # plus any additional origins set via ALLOWED_ORIGINS env var (comma-separated)
24 _default_origins = [
25     "http://localhost:5173",
26     "http://localhost:5174",
27     "http://127.0.0.1:5173",
28     "http://127.0.0.1:5174",
29     "https://finrelief-frontend.onrender.com",
30 ]
31 _extra_origins = os.environ.get("ALLOWED_ORIGINS", "")
32 _extra_list = [o.strip() for o in _extra_origins.split(",") if o]
33 allowed_origins = list(set(_default_origins + _extra_list))
34
35 app.add_middleware(
36     CORSMiddleware,
37     allow_origins=allowed_origins,
38     allow_credentials=True,
39     allow_methods=["*"],
40     allow_headers=["*"],
41 )
42
43 # Register routers under /api prefix
44 app.include_router(auth.router, prefix="/api")
45 app.include_router(debts.router, prefix="/api")
```

Activity 2.2: Implement the FastAPI Backend With Python

The backend was developed using FastAPI to handle business logic and API communication.

The following backend functionalities were implemented:

- Develop REST APIs for user authentication and profile management.
- Implement loan management APIs for storing and retrieving borrower loan information.
- Integrate SQLite database using SQLAlchemy ORM.
- Implement financial analysis calculations including EMI ratio, debt stress analysis, and financial health score.
- Integrate Google Gemini 2.5 Flash API for AI-powered settlement recommendations and negotiation

letter generation.

- Store AI-generated responses and negotiation history in the database.
- Secure all protected routes using JWT Authentication.

```
1 from pydantic import BaseModel, ConfigDict
2 from typing import List, Optional
3 import datetime
4
5 # User Schemas
6 class UserBase(BaseModel):
7     username: str
8     name: Optional[str] = None
9     email: Optional[str] = None
10
11 class UserCreate(UserBase):
12     password: str
13
14 class UserResponse(UserBase):
15     user_id: int
16     username: str
17     name: Optional[str] = None
18     email: Optional[str] = None
19     monthly_income: float
20     monthly_expenses: float
21     extra_payment: float
22     strategy: str
23     onboarding_completed: bool
24
25     model_config = ConfigDict(from_attributes=True)
26
27 # Token Schemas
28 class Token(BaseModel):
29     access_token: str
30     token_type: str
31
32 class TokenData(BaseModel):
33     username: Optional[str] = None
34
35 # Debt/Loan Schemas (Combined for maximum compatibility and ERD alignment)
36 class DebtBase(BaseModel):
37     creditor: str
38     balance: float # outstanding_amount
39     apr: float # interest_rate
40     min_payment: float
41     category: str = "Credit Card" # loan_type
42     emi: Optional[float] = None
43     overdue_months: int = 0
44     loan_amount: float = 0.0 # original loan amount (new ERD column)
45     due_date: Optional[str] = None # payment due date (new ERD column)
```

Milestone 3: Backend Development

Milestone 3 focuses on developing and refining the core logic of the FastAPI backend, which serves as the backbone of the **AI Powered Debt Relief & Financial Recovery Platform**. In this milestone, the backend APIs are implemented for user authentication, loan management, financial analysis, AI-powered settlement recommendations, negotiation letter generation, database operations, and history management. The backend also integrates Google Gemini 2.5 Flash API to generate intelligent financial guidance and personalized negotiation strategies..

Activity 3.1 Writing the Main Backend Application

Define the Core API Routes

Establish REST API routes for the platform, where each endpoint performs a specific operation.

- **POST /register** – Registers a new borrower account.
- **POST /login** – Authenticates the borrower and generates a JWT access token.
- **GET /profile** – Retrieves the borrower profile information.
- **POST /loans** – Adds new loan details.
- **PUT /loans/{id}** – Updates an existing loan.
- **DELETE /loans/{id}** – Deletes a loan record.
- **GET /loans** – Retrieves all loan details.
- **POST /financial-analysis** – Calculates EMI-to-Income Ratio, Monthly Surplus, Debt Stress Level, and Financial Health Score.
- **POST /ai/generate-letter** – Generates AI-powered negotiation letters using Google Gemini 2.5 Flash.
- **GET /history** – Retrieves previously generated AI negotiation history.

```
1 import os
2 from fastapi import FastAPI
3 from fastapi.middleware.cors import CORSMiddleware
4 from dotenv import load_dotenv
5
6 # Load environmental variables first
7 load_dotenv()
8
9 from app.database import engine
10 from app import models
11 from app.routers import auth, debts, profile, ai
12
13 # Initialize DB tables (SQLite auto-bootstrap)
14 models.Base.metadata.create_all(bind=engine)
15
16 app = FastAPI(
17     title="FinRelief AI Backend",
18     description="Secure backend service for debt recovery and analysis modules, authenticated via JWT and integrated with Google Gemini 2.5 Flash",
19     version="1.0.0"
20 )
21
22 # Build allowed origins: always include localhost dev origins,
23 # plus any additional origins set via ALLOWED_ORIGINS env var (comma-separated)
24 _default_origins = [
25     "http://localhost:5173",
26     "http://localhost:5174",
27     "http://127.0.0.1:5173",
28     "http://127.0.0.1:5174",
29     "https://finrelief-frontend.onrender.com",
30 ]
31 _extra_origins = os.environ.get("ALLOWED_ORIGINS", "")
32 _extra_list = [o.strip() for o in _extra_origins.split(",") if o]
33 allowed_origins = list(set(_default_origins + _extra_list))
34
35 app.add_middleware(
36     CORSMiddleware,
37     allow_origins=allowed_origins,
38     allow_credentials=True,
39     allow_methods=["*"],
40     allow_headers=["*"],
41 )
42
43 # Register routers under /api prefix
44 app.include_router(auth.router, prefix="/api")
45 app.include_router(debts.router, prefix="/api")
```

Process User Requests

- Receive borrower information from the React frontend.
- Validate all incoming requests using Pydantic schemas.
- Process loan details and financial information.
- Store validated information securely in the SQLite database.

Implement Database Operations

- Configure SQLite using SQLAlchemy ORM.
- Store user profiles.
- Store loan information.
- Store financial profiles.
- Store settlement records.
- Store AI-generated negotiation history.

```
9  import sqlite3
10  conn = sqlite3.connect('finrelief.db')
11  cur = conn.cursor()
12  cur.execute("SELECT name FROM sqlite_master WHERE type='table' ORDER BY name")
13  tables = [r[0] for r in cur.fetchall()]
14  print("Tables in database:", tables)
15
16  # Verify User columns
17  cur.execute("PRAGMA table_info(users)")
18  user_cols = [r[1] for r in cur.fetchall()]
19  print("Users table columns:", user_cols)
20
21  # Verify FinancialProfile columns
22  cur.execute("PRAGMA table_info(financial_profiles)")
23  profile_cols = [r[1] for r in cur.fetchall()]
24  print("FinancialProfile table columns:", profile_cols)
25
26  # Verify Loan columns
27  cur.execute("PRAGMA table_info(loans)")
28  loan_cols = [r[1] for r in cur.fetchall()]
29  print("Loans table columns:", loan_cols)
30
31  # Verify SettlementRecord columns
32  cur.execute("PRAGMA table_info(settlement_records)")
33  settlement_cols = [r[1] for r in cur.fetchall()]
34  print("SettlementRecord table columns:", settlement_cols)
35
36  # Verify AIHistory columns
37  cur.execute("PRAGMA table_info(ai_histories)")
38  ai_cols = [r[1] for r in cur.fetchall()]
39  print("AIHistory table columns:", ai_cols)
40
41  conn.close()
42  print("All Schema Verification Checks: OK")
43
```

Integrate Google Gemini API

- Generate personalized settlement recommendations.
- Generate Interest Rate Reduction Request letters.
- Generate Debt Validation Request letters.
- Generate Partial Payment Settlement letters.
- Return AI responses in JSON format through FastAPI.

Secure the Backend

- Implement JWT Authentication.
- Configure password hashing.
- Protect private API endpoints.
- Validate requests and handle exceptions.
- Configure CORS middleware.

```
1 from pydantic import BaseModel, ConfigDict
2 from typing import List, Optional
3 import datetime
4
5 # User Schemas
6 class UserBase(BaseModel):
7     username: str
8     name: Optional[str] = None
9     email: Optional[str] = None
10
11 class UserCreate(UserBase):
12     password: str
13
14 class UserResponse(UserBase):
15     user_id: int
16     username: str
17     name: Optional[str] = None
18     email: Optional[str] = None
19     monthly_income: float
20     monthly_expenses: float
21     extra_payment: float
22     strategy: str
23     onboarding_completed: bool
24
25     model_config = ConfigDict(from_attributes=True)
26
27 # Token Schemas
28 class Token(BaseModel):
29     access_token: str
30     token_type: str
31
32 class TokenData(BaseModel):
33     username: Optional[str] = None
34
35 # Debt/Loan Schemas (Combined for maximum compatibility and ERD alignment)
36 class DebtBase(BaseModel):
37     creditor: str
38     balance: float # outstanding_amount
39     apr: float # interest_rate
40     min_payment: float
41     category: str = "Credit Card" # loan_type
42     emi: Optional[float] = None
43     overdue_months: int = 0
44     loan_amount: float = 0.0 # original loan amount (new ERD column)
45     due_date: Optional[str] = None # payment due date (new ERD column)
```

Testing the Backend

Test all backend modules to verify:

- User Authentication
- Loan Management
- Financial Analysis
- AI Settlement Recommendation
- Negotiation Letter Generation
- Database Connectivity
- REST API Responses

Milestone 4: Frontend Development

Milestone 4 focuses on designing and developing a responsive, user-friendly, and interactive interface for the **AI Powered Debt Relief & Financial Recovery Platform**. This milestone involves building the frontend using **React.js, Vite, Tailwind CSS, and Context API**, creating reusable user interface components, and implementing dynamic communication with the FastAPI backend through REST APIs. The frontend enables borrowers to securely manage loan information, monitor financial health, receive AI-powered settlement recommendations, generate negotiation letters, and track previous AI-generated history.

Activity 4.1: Designing and Developing the User Interface

Set Up the React Application

- Develop the frontend using React.js and Vite to provide a fast and efficient development environment.
- Organize the project using reusable React components, pages, services, and context management.
- Configure React Router for smooth navigation between different pages.

```
1  /** @type {import('tailwindcss').Config} */
2  export default {
3    content: [
4      "./index.html",
5      "./src/**/*..{js,ts,jsx,tsx}",
6    ],
7    theme: {
8      extend: {},
9    },
10   plugins: [],
11 }
```

Design a Responsive User Interface

- Design the interface using Tailwind CSS to create a clean, modern, and responsive layout.
- Use responsive layouts to support desktop, tablet, and mobile devices.
- Organize the application using cards, forms, tables, navigation bars, and dashboard components.

```

1  {
2    "name": "finrelief-frontend",
3    "version": "0.0.0",
4    "lockfileVersion": 3,
5    "requires": true,
6    "packages": {
7      "": {
8        "name": "finrelief-frontend",
9        "version": "0.0.0",
10       "dependencies": {
11         "lucide-react": "^1.22.0",
12         "react": "^19.2.7",
13         "react-dom": "^19.2.7",
14         "recharts": "^3.9.0"
15       },
16       "devDependencies": {
17         "@eslint/js": "^10.0.1",
18         "@tailwindcss/vite": "^4.3.2",
19         "@types/react": "^19.2.17",
20         "@types/react-dom": "^19.2.3",
21         "@vitejs/plugin-react": "^6.0.2",
22         "autoprefixer": "^10.5.2",
23         "eslint": "^10.5.0",
24         "eslint-plugin-react-hooks": "^7.1.1",
25         "eslint-plugin-react-refresh": "^0.5.3",
26         "globals": "^17.6.0",
27         "postcss": "^8.5.16",
28         "tailwindcss": "^4.3.2",
29         "vite": "^8.1.0"
30       }
31     },
32     "node_modules/@babel/code-frame": {
33       "version": "7.29.7",
34       "resolved": "https://registry.npmjs.org/@babel/code-frame/-/code-frame-7.29.7.tgz",
35       "integrity": "sha512-AU0v8ZCDBsZqS3kR/0fBnA8zDtZTQWz4vF5zqQ3kpSlzWdhgNm/ztIzDhIF5igBNWyd4j/94vYU6WvD4Zg==",
36       "dev": true,
37       "license": "MIT",
38       "dependencies": {
39         "@babel/helper-validator-identifier": "^7.29.7",
40         "js-tokens": "^4.0.0",
41         "picocolors": "^1.1.1"
42       },
43       "engines": {
44         "node": ">=6.9.0"
45     }
  }

```

Create User Interface Components

- Authentication Pages: Login and Registration.
- Dashboard: Financial Health Summary.
- Loan Management: Add, Update, Delete, and View Loans.
- Financial Analysis: EMI Ratio, Debt Stress Level, Monthly Surplus.
- AI Recommendation: Settlement Recommendation.
- Negotiation Letter: AI Letter Generator.
- History: AI Negotiation History.

Activity 4.2: Creating Dynamic User Interfaces

Handle User Requests

- Send requests from React to FastAPI using REST APIs.
- Validate user input before submitting forms.
- Display loading indicators while processing AI responses.

```
1  {
2    "name": "finrelief-frontend",
3    "private": true,
4    "version": "0.0.0",
5    "type": "module",
6    > Debug
7    "scripts": {
8      "dev": "vite",
9      "build": "vite build",
10     "lint": "eslint .",
11     "preview": "vite preview"
12   },
13   "dependencies": {
14     "lucide-react": "^1.22.0",
15     "react": "^19.2.7",
16     "react-dom": "^19.2.7",
17     "recharts": "^3.9.0"
18   },
19   "devDependencies": {
20     "@eslint/js": "^10.0.1",
21     "@tailwindcss/vite": "^4.3.2",
22     "@types/react": "^19.2.17",
23     "@types/react-dom": "^19.2.3",
24     "@vitejs/plugin-react": "^6.0.2",
25     "autoprefixer": "^10.5.2",
26     "eslint": "^10.5.0",
27     "eslint-plugin-react-hooks": "^7.1.1",
28     "eslint-plugin-react-refresh": "^0.5.3",
29     "globals": "^17.6.0",
30     "postcss": "^8.5.16",
31     "tailwindcss": "^4.3.2",
32     "vite": "^8.1.0"
33   }
34 }
```

Render Dynamic Data

- Display borrower loan information dynamically.
- Show financial health metrics.
- Display AI-generated settlement recommendations.
- Render negotiation letters generated by Google Gemini.
- Display negotiation history stored in the database.

Update the User Interface

- Refresh dashboard information after every update.
- Display success and error notifications.
- Synchronize frontend data with backend responses in real time.

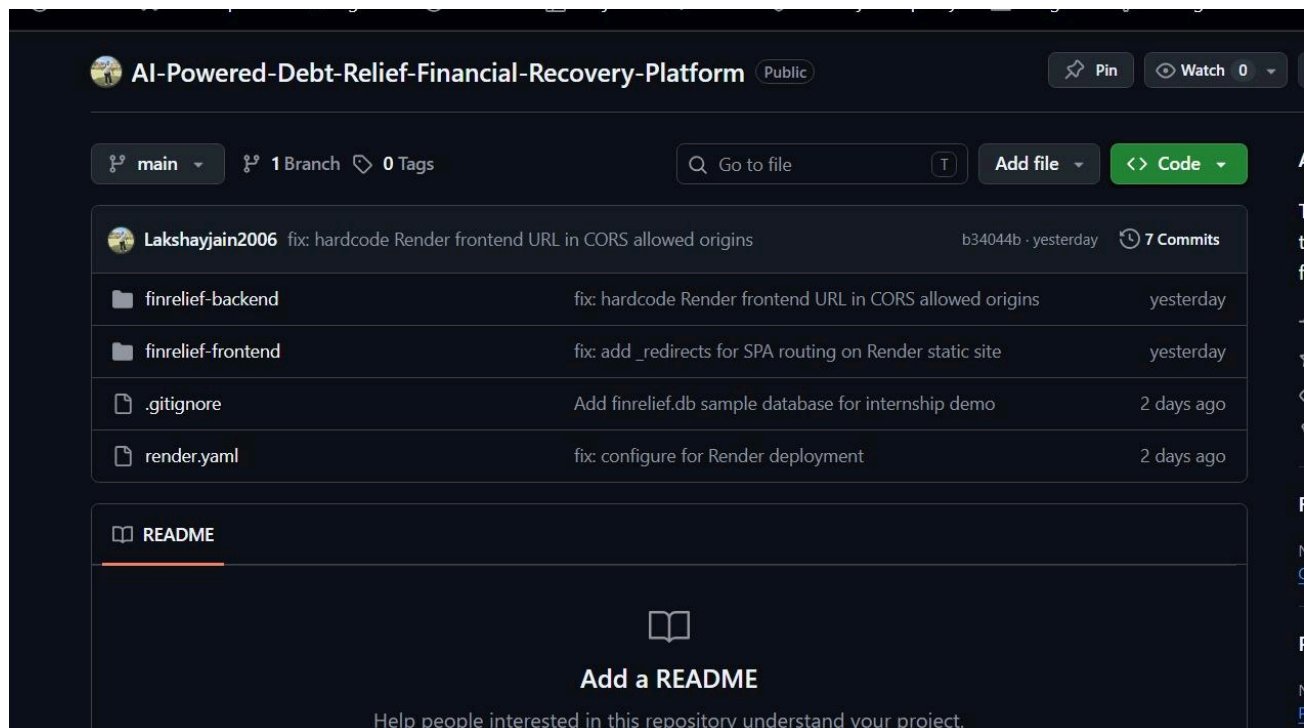
Milestone 5: Deployment

This milestone focuses on deploying the AI Powered Debt Relief & Financial Recovery Platform into a production-ready environment. The deployment process includes configuring the frontend, backend, database, environment variables, and production settings. After successful configuration, the React frontend is deployed and connected with the FastAPI backend, ensuring secure communication between the application, SQLite database, and Google Gemini API.

Activity 5.1: Prepare the Application for Deployment

Prepare the Backend Environment

The backend application is prepared by organizing the project structure, installing all required dependencies, configuring the virtual environment, and verifying that the FastAPI server is ready for deployment.



Configure Environment Variables

Configure all required environment variables before deployment. The database URL, JWT secret key, and Google Gemini API key are stored securely in the **.env** file, while unnecessary files are excluded using the **.gitignore** file.

```
1  # SQLite (local dev) or PostgreSQL (production)
2  DATABASE_URL=sqlite:///./finrelief.db
3
4  # JWT signing key - generate a strong random string for production
5  # Example: python -c "import secrets; print(secrets.token_hex(32))"
6  JWT_SECRET_KEY=your_jwt_secret_key_here
7
8  # Google Gemini API key - get from https://aistudio.google.com/
9  GEMINI_API_KEY=your_gemini_api_key_here
10
```

Verify Project Configuration

Verify the deployment configuration by checking the render.yaml file, project structure, build commands, environment variables, and backend service settings before deploying the application.

```
1  # render.yaml – Render Blueprint for FinRelief
2  # Docs: https://render.com/docs/blueprint-spec
3  #
4  # HOW TO USE:
5  #   1. Push this repo to GitHub
6  #   2. Go to https://dashboard.render.com/ → New → Blueprint
7  #   3. Connect your repo – Render will auto-detect this file
8  #   4. Fill in the secret env vars (GEMINI_API_KEY, JWT_SECRET_KEY) in the dashboard
9
10 services:
11   # — Backend: FastAPI —————
12   - type: web
13     name: finrelief-backend
14     runtime: python
15     rootDir: finrelief-backend
16     buildCommand: pip install -r requirements.txt
17     startCommand: uvicorn app.main:app --host 0.0.0.0 --port $PORT
18     plan: free
19     envVars:
20       - key: GEMINI_API_KEY
21         sync: false # You will paste this in the Render dashboard
22       - key: JWT_SECRET_KEY
23         generateValue: true # Render auto-generates a secure random value
24       - key: ALLOWED_ORIGINS
25         # Set this AFTER frontend deploys. Format: https://finrelief-frontend.onrender.com
26         sync: false
27
28   # — Frontend: Vite/React Static Site —————
29   - type: web
30     name: finrelief-frontend
31     runtime: static
32     rootDir: finrelief-frontend
33     buildCommand: npm install && npm run build
34     staticPublishPath: dist
35     plan: free
36     routes:
37       # SPA fallback: all routes serve index.html
38       - type: rewrite
39         source: /*
40         destination: /index.html
41     envVars:
42       - key: VITE_API_URL
43         # Set this to your backend URL: https://finrelief-backend.onrender.com
44         sync: false
45
```

Prepare the Deployment Environment

Open the Visual Studio Code terminal and verify that all frontend and backend dependencies are installed successfully before deployment.

```

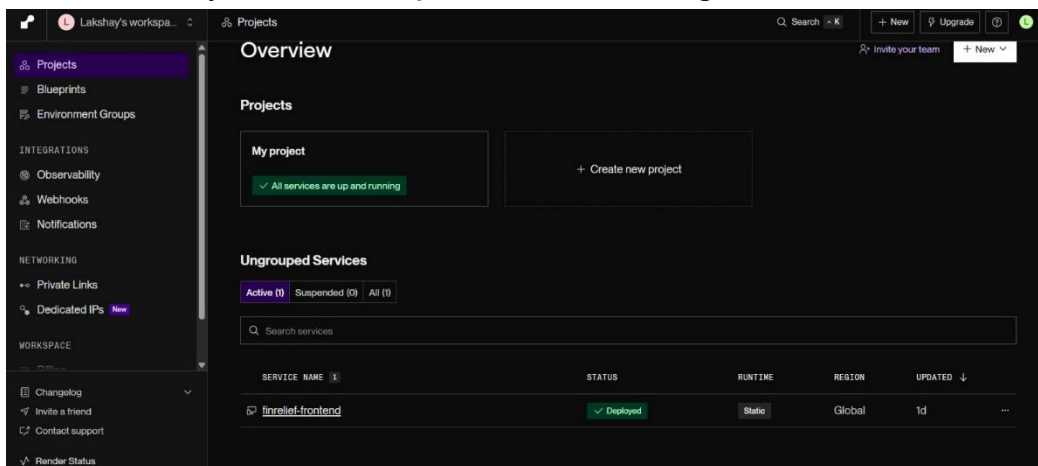
1  # SQLite (local dev) or PostgreSQL (production)
2  DATABASE_URL=sqlite:///./finrelief.db
3
4  # JWT signing key - generate a strong random string for production
5  # Example: python -c "import secrets; print(secrets.token_hex(32))"
6  JWT_SECRET_KEY=your_jwt_secret_key_here
7
8  # Google Gemini API key - get from https://aistudio.google.com/
9  GEMINI_API_KEY=your_gemini_api_key_here
10

```

Activity 5.2: Deploy the Application

Deploy the Frontend and Backend

Deploy the React frontend and FastAPI backend using the configured deployment platform. Ensure that the frontend communicates successfully with the backend through REST APIs, while the backend securely accesses the SQLite database and Google Gemini API.



Verify Application Deployment

After deployment, verify that all major functionalities are working correctly.

- User Registration and Login
- Loan Management
- Financial Health Analysis
- AI Settlement Recommendation
- AI Negotiation Letter Generation
- Database Connectivity
- Frontend and Backend Communication

- Google Gemini API Integration

Production Deployment

The application is successfully deployed with the following configuration:

- Frontend: React.js (Production Build)
- Backend: FastAPI
- Database: SQLite
- AI Integration: Google Gemini 2.5 Flash
- Authentication: JWT Authentication
- Deployment Platforms: Render and Vercel

Exploring the Website's Web Pages:

Home / Generator Page:



Description: The **home page** is the main page of the platform. It provides an overview of the borrower's financial condition by displaying total liabilities, EMI ratio, debt-to-income ratio, monthly surplus, recovery score, repayment outlook, and liability breakdown. It also provides quick navigation to all major modules of the application.

Input Section:

Add Creditor Account

Cancel

CREDITOR / ACCOUNT NAME

BALANCE (₹)

INTEREST (APR %)

 axis shoothing card

25000

10

MINIMUM MONTHLY PAYMENT (₹)

ACCOUNT TYPE

4500

Credit Card

MONTHLY EMI (₹) — for installment loans

MONTHS OVERDUE — 0 if current

e.g. 12000 (leave blank if N/A)

0

ORIGINAL LOAN AMOUNT (₹) — default: Balance

PAYMENT DUE DATE

e.g. 200000

mm/dd/yyyy

Add Account To Ledger

Description: The **Input Page** allows borrowers to enter and manage their loan information. Users can provide creditor details, outstanding balance, interest rate, monthly payment, EMI, overdue months, original loan amount, and payment due date. The submitted information is stored securely and used for financial analysis and AI-powered recommendations.

Results Section:

Liability Ledger
 List of all active credit accounts, loans, and balances.

+ Add Debt

CREDITOR	CATEGORY	LOAN AMT	BALANCE	APR	MIN PAYMENT	DUE DATE	ACTIONS
sbi credit card	Credit Card	₹10,000	₹10,000	15%	₹3600	N/A	 
axis card	Credit Card	₹100,000	₹150,000	19.9%	₹4000	N/A	 
axis shoothing card	Credit Card	₹25,000	₹25,000	10%	₹4500	N/A	 

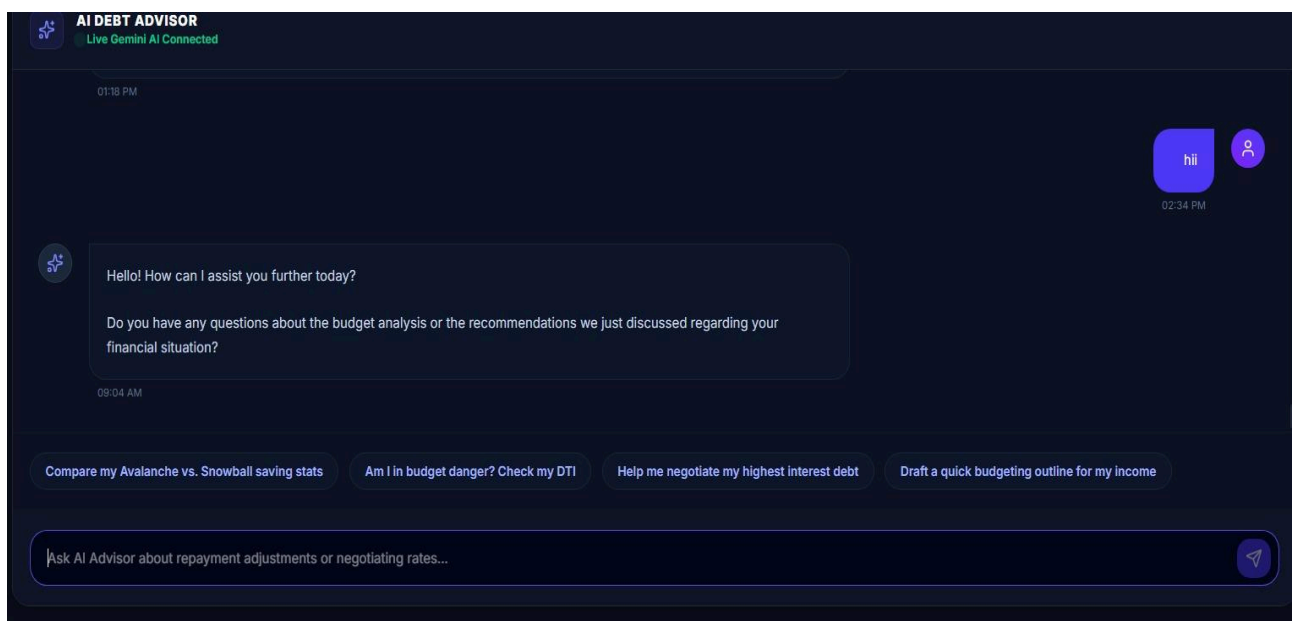
Description :

The Result Page displays all saved loan accounts in a structured table format. Borrowers can view creditor information, loan amount, balance, APR, minimum payment, and due dates. The page also provides options to edit or delete loan records, helping users efficiently manage their debt portfolio.

Feature:

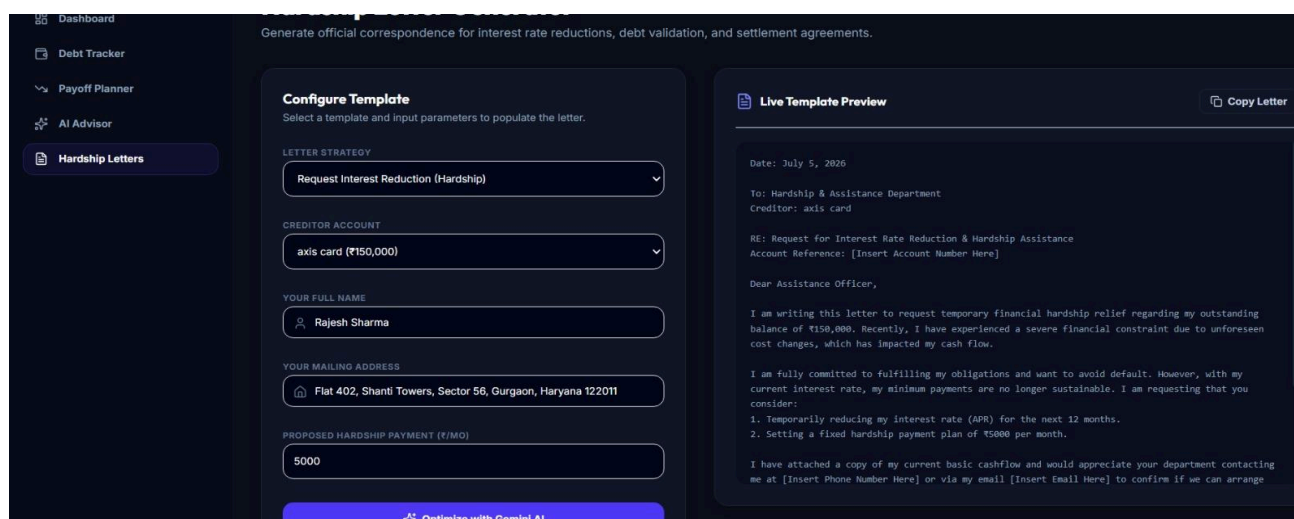
- **AI Debt Advisor :-**

The **AI Debt Advisor** is an AI-powered assistant that helps users manage their debt and improve their financial situation. Users can ask questions about loan repayment, budgeting, debt settlement, and financial planning. Based on the user's loan details and financial information, the AI provides simple, personalized suggestions and useful guidance. It also offers quick answers to common financial questions, making the platform easier to use and helping users make better financial decisions.



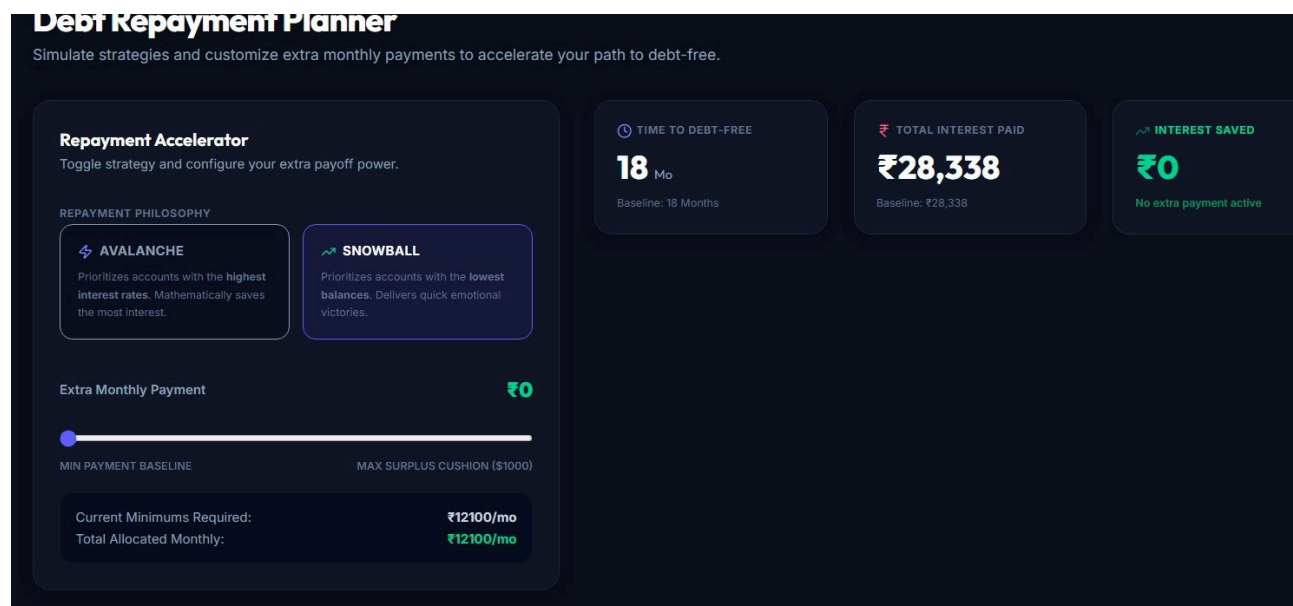
- **AI Hardship Letter Generator :-**

The AI Hardship Letter Generator helps borrowers create professional negotiation letters for their lenders. Users can select a letter type, choose a loan account, and enter the required details such as their name, address, and proposed payment amount. Using Google Gemini AI, the system generates a personalized hardship or settlement letter that can be reviewed, copied, and sent to the lender. This feature saves time, improves communication, and helps borrowers request better repayment terms in a simple and professional way.



Debt Repayment Planner:-

The Debt Repayment Planner helps borrowers create a clear repayment plan based on their loan details and monthly payment capacity. Users can choose between the Avalanche and Snowball repayment strategies and adjust extra monthly payments to compare different repayment scenarios. The system calculates the estimated time to become debt-free, total interest to be paid, and potential interest savings. It also displays a payoff projection graph and a month-by-month repayment schedule, helping borrowers understand their repayment progress and choose the most suitable strategy for achieving financial recovery.



Conclusion:

The **AI Powered Debt Relief & Financial Recovery Platform** is designed to provide borrowers with a smart, secure, and user-friendly solution for managing debt and improving their financial well-being. The platform enables users to add and organize loan details, monitor financial health, calculate debt-related metrics, and receive personalized settlement recommendations based on their financial condition. It also generates AI-powered negotiation letters that help borrowers communicate professionally with lenders and request better repayment terms.

The application is built using **React.js**, **FastAPI**, **SQLite**, **SQLAlchemy ORM**, and **Google Gemini AI**, creating a modern and efficient system for financial recovery. Features such as debt tracking, repayment planning, financial health analysis, AI Debt Advisor, and negotiation history provide users with valuable insights and practical guidance throughout their debt repayment journey. The responsive interface makes the platform easy to use across different devices while ensuring secure handling of financial information through JWT authentication and structured database management.

Overall, this project demonstrates how Artificial Intelligence can simplify debt management, reduce financial stress, and support informed financial decision-making. By combining automation, real-time analysis, and personalized AI assistance, the platform helps borrowers work toward long-term financial recovery, better money management, and greater financial stability with confidence.